

TÀI LIỆU KHẢO SÁT LÝ THUYẾT & CHẠY TỪNG BƯỚC GIẢI THUẬT ĐỒ THỊ

Hệ thống giáo trình trực quan hỗ trợ học tập môn Cấu trúc rời rạc / Lý thuyết đồ thị

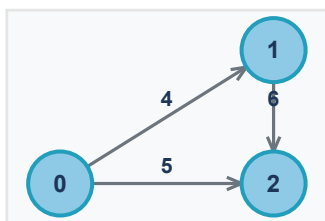
1. Các phương pháp biểu diễn đồ thị và Chuyển đổi

Khái niệm & Định nghĩa toán học: Đồ thị $G = (V, E)$ là một cấu trúc toán học gồm tập hợp các đỉnh V và tập hợp các cạnh E nối các cặp đỉnh thuộc V . Đồ thị có thể là vô hướng (các cạnh không phân biệt hướng) hoặc có hướng (mỗi cạnh là một cặp đỉnh có thứ tự từ đỉnh đầu đến đỉnh cuối, ký hiệu là digraph). Đồ thị có trọng số là đồ thị mà mỗi cạnh được gán một giá trị số thực thể hiện chi phí, khoảng cách hoặc năng lực thông qua.

Để xử lý đồ thị bằng máy tính, ta sử dụng 3 phương pháp biểu diễn cơ bản:

- Ma trận kề (Adjacency Matrix):** Mảng hai chiều kích thước $|V| \times |V|$. Ô $A[i][j]$ lưu giá trị trọng số cạnh (hoặc 1 nếu không trọng số) nếu có cạnh nối từ i đến j , ngược lại lưu 0 hoặc vô cực. Độ phức tạp không gian: $O(V^2)$. Phù hợp cho đồ thị dày, kiểm tra kết nối nhanh $O(1)$ nhưng tốn bộ nhớ với đồ thị thưa.
- Danh sách kề (Adjacency List):** Danh sách chứa các đỉnh kề trực tiếp của từng đỉnh. Độ phức tạp không gian: $O(V + E)$. Tiết kiệm bộ nhớ tối đa cho đồ thị thưa, nhưng kiểm tra kết nối hai đỉnh mất thời gian $O(\text{bậc của đỉnh})$.
- Danh sách cạnh (Edge List):** Danh sách lưu trữ trực tiếp các bộ ba (u, v, w) tương ứng với cạnh nối u, v có trọng số w . Độ phức tạp không gian: $O(E)$. Cực kỳ tiện lợi cho các giải thuật duyệt cạnh trực tiếp như Kruskal.

Đồ thị mẫu (3 đỉnh):



Ma trận kề (Adjacency Matrix):

	0	1	2
0	0	4	5
1	0	0	6
2	0	0	0

Danh sách kề (Adjacency List):

Đỉnh	Danh sách kề (trọng số)
0	1 (w:4), 2 (w:5)
1	2 (w:6)
2	-

Danh sách cạnh (Edge List):

Đầu	Cuối	Trọng số
0	1	4
0	2	5
1	2	6

2. Giải thuật duyệt đồ thị: BFS và DFS

Định nghĩa giải thuật: Duyệt đồ thị là quá trình đi qua tất cả các đỉnh của đồ thị một cách hệ thống, đảm bảo mỗi đỉnh được duyệt đúng một lần.

- BFS (Breadth-First Search - Duyệt theo chiều rộng): Xuất phát từ đỉnh nguồn S, thuật toán loang đều ra xung quanh bằng cách duyệt qua tất cả các đỉnh có khoảng cách k cạnh trước khi chuyển sang các đỉnh kề có khoảng cách k+1 cạnh. Giải thuật sử dụng cấu trúc dữ liệu Hàng đợi (Queue - FIFO) để lưu trữ các đỉnh đang chờ xét kề.
- DFS (Depth-First Search - Duyệt theo chiều sâu): Xuất phát từ đỉnh nguồn S, thuật toán ưu tiên đi xa nhất có thể dọc theo mỗi nhánh của đồ thị trước khi thực hiện quay lui (Backtracking). Giải thuật sử dụng Ngăn xếp (Stack - LIFO) (hoặc cơ chế gọi hàm đệ quy của hệ thống).

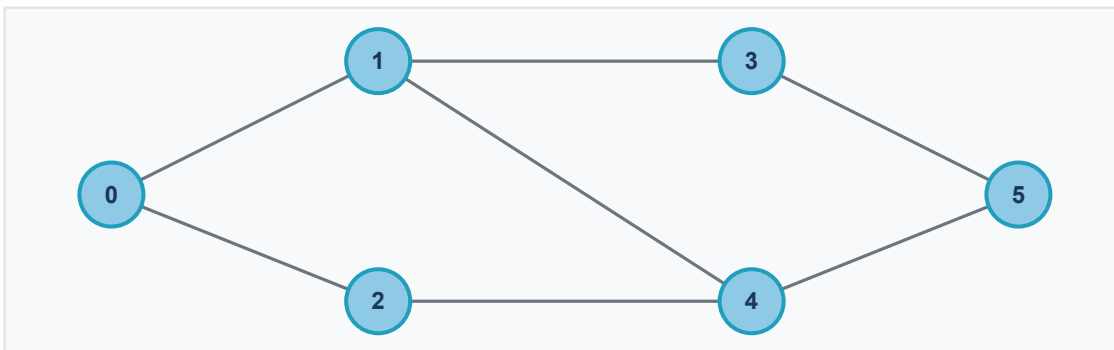
Độ phức tạp giải thuật:

- Thời gian: $O(V + E)$ khi biểu diễn bằng danh sách kề, do mỗi đỉnh được đưa vào Queue/Stack đúng 1 lần và mỗi cạnh được xét tối đa 2 lần (với đồ thị vô hướng). Với ma trận kề, chi phí duyệt kề là $O(V^2)$.
- Không gian phụ trợ: $O(V)$ để lưu trữ mảng đánh dấu 'visited' và hàng đợi/ngăn xếp.

Nhận xét & So sánh thực tế:

- Đồ thị không trọng số: BFS luôn đảm bảo tìm được đường đi ngắn nhất (ít cạnh nhất) từ nguồn tới đích vì nó loang đều theo từng tầng cạnh. DFS không có tính chất này và có thể đi vòng rất xa trước khi tới đích.
- Đồ thị có trọng số: Cả BFS và DFS đều không thể tìm được đường đi ngắn nhất do chi phí đi qua một cạnh không còn đồng nhất. Khi đó bắt buộc phải dùng thuật toán chuyên dụng như Dijkstra hoặc Bellman-Ford.
- Ứng dụng thực tế: BFS được dùng nhiều trong thuật toán định tuyến mạng internet, gợi ý kết nối mạng xã hội, loang màu. DFS dùng nhiều để phát hiện chu trình, phân tích liên thông mạnh, lập lịch topo công việc hoặc giải các bài toán mê cung/quay lui.

Minh họa đồ thị vô hướng khảo sát duyệt BFS & DFS:



Bảng chạy từng bước giải thuật BFS & DFS (bắt đầu từ đỉnh 0):

Bư ớc	Giải thuật BFS (Bắt đầu từ 0)	Hàng đợi (Queue)	Giải thuật DFS (Bắt đầu từ 0)	Ngăn xếp (Stack)
0	Khởi tạo: Thăm 0	[0]	Khởi tạo: Thăm 0	[0]
1	Lấy 0 ra. Thăm đỉnh kề: 1, 2	[1, 2]	Lấy 0 ra. Thăm đỉnh kề: 1 (đi sâu)	[1]
2	Lấy 1 ra. Thăm đỉnh kề: 3, 4	[2, 3, 4]	Lấy 1 ra. Thăm đỉnh kề: 3 (đi sâu)	[3]
3	Lấy 2 ra. Đỉnh kề của 2 (4) đã được xét	[3, 4]	Lấy 3 ra. Thăm đỉnh kề: 5 (đi sâu)	[5]
4	Lấy 3 ra. Thăm đỉnh kề: 5	[4, 5]	Lấy 5 ra. Thăm đỉnh kề: 4 (đi sâu)	[4]
5	Lấy 4 ra. Đỉnh kề đã xét	[5]	Lấy 4 ra. Thăm đỉnh kề: 2 (đi sâu)	[2]
6	Lấy 5 ra. Hàng đợi rỗng. Kết thúc.	[]	Lấy 2 ra. Không còn kề chưa thăm. Quay lui.	[]
KQ	Thứ tự duyệt BFS: 0, 1, 2, 3, 4, 5	-	Thứ tự duyệt DFS: 0, 1, 3, 5, 4, 2	-

3. Kiểm tra đồ thị hai phía (Bipartite Graph Check)

Khái niệm & Định nghĩa toán học: Một đồ thị vô hướng $G = (V, E)$ được gọi là đồ thị hai phía (Bipartite Graph) nếu tập đỉnh V của nó có thể phân hoạch thành hai tập con độc lập V_1 và V_2 ($V_1 \cap V_2$ bằng rỗng, $V_1 \cup V_2$ bằng V) sao cho mọi cạnh trong E đều kết nối một đỉnh thuộc V_1 với một đỉnh thuộc V_2 . Điều này nghĩa là không tồn tại cạnh nào nối giữa các đỉnh trong cùng một tập con.

Định lý quyết định: Một đồ thị là đồ thị hai phía khi và chỉ khi đồ thị đó không chứa chu trình lẻ (chu trình có số cạnh là số lẻ như $C_3, C_5, \dots$).

Nguyên lý giải thuật (Tô màu đồ thị - 2-Coloring):

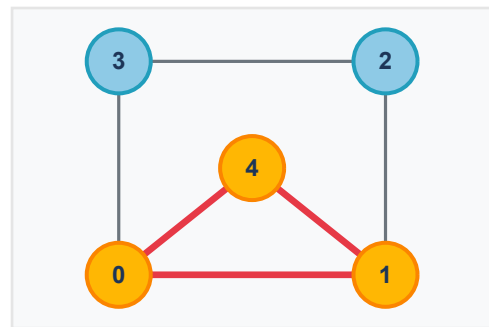
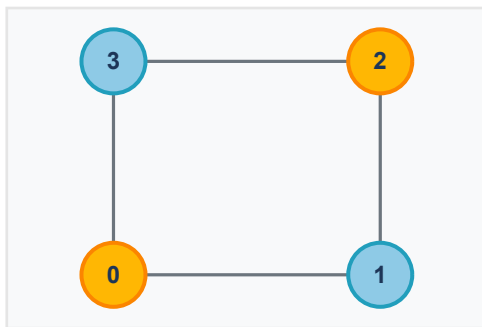
Thuật toán cố gắng tô tất cả các đỉnh của đồ thị bằng 2 màu (ví dụ Vàng và Xanh) sao cho không có hai đỉnh kề nhau nào có cùng màu. Ta khởi tạo tất cả đỉnh chưa có màu, chọn một đỉnh xuất phát tô màu Vàng (1), đưa vào Queue.

Thực hiện loang BFS: Với mỗi đỉnh u lấy ra từ Queue, ta kiểm tra các đỉnh kề v của nó:

- Nếu v chưa tô màu: Tô màu v ngược với màu của u (Xanh) và đưa v vào Queue.
- Nếu v đã được tô màu: Kiểm tra màu của v . Nếu màu của v trùng với màu của u , lập tức kết luận đồ thị không phải hai phía và dừng thuật toán.

Độ phức tạp thuật toán: $O(V + E)$ thời gian và $O(V)$ không gian, tương đương một lượt duyệt BFS đầy đủ.

Minh họa tô 2 màu trên đồ thị 2 phía (C_4) và đồ thị chứa chu trình lẻ (C_5):



Bên trái là đồ thị hai phía (C_4) tô màu xen kẽ thành công (màu vàng/xanh lam). Bên phải là đồ thị chứa chu trình lẻ 0-1-4 (không phải hai phía) được highlight cạnh đỏ mâu thuẫn màu.

Bảng chạy từng bước thuật toán Tô 2 màu kiểm tra Đồ thị hai phía:

TH1: Chạy vết với Đồ thị hai phía C4 (bắt đầu từ đỉnh 0):

Bước c	Xét U	Màu U	Đỉnh kề V của U	Trạng thái tô màu đỉnh kề V	Hàng đợi (Queue)
0	-	-	-	Khởi tạo tô đỉnh 0: Vàng (1)	[0]
1	0	Vàng	1, 3	Tô 1: Xanh (2); Tô 3: Xanh (2)	[1, 3]
2	1	Xanh	0, 2	0: Vàng (đã tô); Tô 2: Vàng (1)	[3, 2]
3	3	Xanh	0, 2	0: Vàng (khớp màu); 2: Vàng (khớp màu)	[2]
4	2	Vàng	1, 3	1: Xanh (khớp màu); 3: Xanh (khớp màu)	[]
KQ	-	-	-	Không xung đột màu. Đồ thị là HAI PHÍA.	[]

TH2: Chạy vết với Đồ thị không hai phía C5 (bắt đầu từ đỉnh 0):

Bước c	Xét U	Màu U	Đỉnh kề V của U	Trạng thái tô màu đỉnh kề V	Hàng đợi (Queue)
0	-	-	-	Khởi tạo tô đỉnh 0: Vàng (1)	[0]
1	0	Vàng	1, 3, 4	Tô 1: Xanh; Tô 3: Xanh; Tô 4: Xanh	[1, 3, 4]
2	1	Xanh	0, 2, 4	0: Vàng; Tô 2: Vàng; 4: Xanh (Đỉnh kề 4 đã có màu Xanh trùng với màu của 1!)	Dừng thuật toán
KQ	-	-	-	Cạnh (1, 4) kết nối 2 đỉnh cùng màu Xanh -> KHÔNG PHẢI HAI PHÍA.	-

4. Thuật toán tìm đường đi ngắn nhất Dijkstra

Khái niệm toán học & Bài toán: Cho đồ thị vô hướng hoặc có hướng liên thông có trọng số không âm $G = (V, E, w)$, trong đó $w(u, v) \geq 0$ là chi phí đi trên cạnh (u, v) . Thuật toán Dijkstra dùng để tìm đường đi ngắn nhất xuất phát từ một đỉnh nguồn S cố định tới tất cả các đỉnh còn lại của đồ thị.

Nguyên lý hoạt động (Kỹ thuật Tham lam - Greedy & Tối ưu hóa - Relaxation):

Giải thuật duy trì một mảng khoảng cách tạm thời $d[v]$ lưu khoảng cách ngắn nhất từ nguồn S tới v tìm thấy cho tới nay. Khởi tạo $d[S] = 0$ và $d[v] = \infty$ với mọi v khác S . Ở mỗi bước lặp:

- Chọn đỉnh u chưa được cố định có giá trị $d[u]$ nhỏ nhất. Đỉnh u này lúc này được cố định khoảng cách (không bao giờ tối ưu hơn được nữa).
- Cập nhật (Relaxation) cho mọi đỉnh kề v chưa cố định của u : nếu $d[u] + w(u, v) < d[v]$, ta cập nhật $d[v] = d[u] + w(u, v)$ và lưu vết đỉnh trước v là u ($prev[v] = u$).

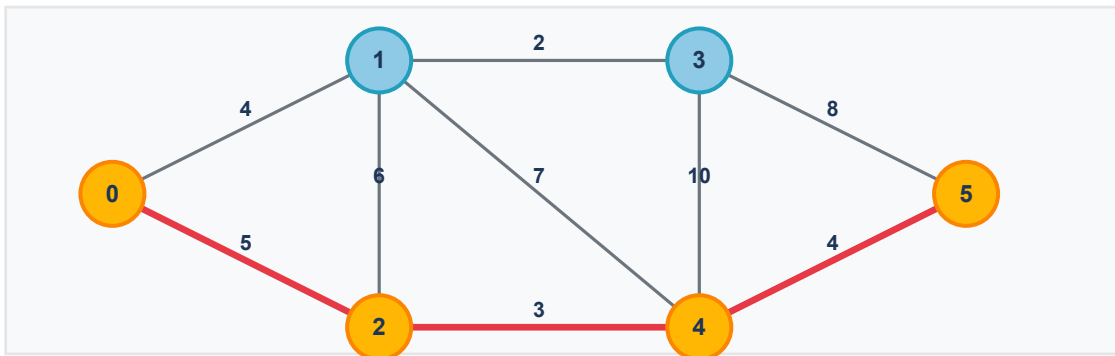
Độ phức tạp thuật toán:

- Dùng mảng thường: $O(V^2)$ thời gian. Phù hợp cho đồ thị dày kề sát nhau.
- Dùng Hàng đợi ưu tiên (Binary Heap / Priority Queue): $O(E \log V)$ thời gian. Cực kỳ tối ưu cho đồ thị thưa.
- Không gian: $O(V)$ để lưu trữ khoảng cách, hàng đợi và vết đường đi.

Tại sao Dijkstra không hoạt động trên đồ thị có cạnh trọng số âm?

Giải thuật dựa trên nguyên lý tham lam: một khi đã cố định đỉnh u có khoảng cách nhỏ nhất, thuật toán coi như đường đi tới u là ngắn nhất và không cập nhật lại. Nếu có cạnh trọng số âm, việc đi vòng qua các cạnh khác có thể mang lại tổng chi phí nhỏ hơn khoảng cách đã cố định, làm sai lệch kết quả tối ưu. Đối với đồ thị có cạnh âm, ta bắt buộc phải dùng thuật toán Bellman-Ford.

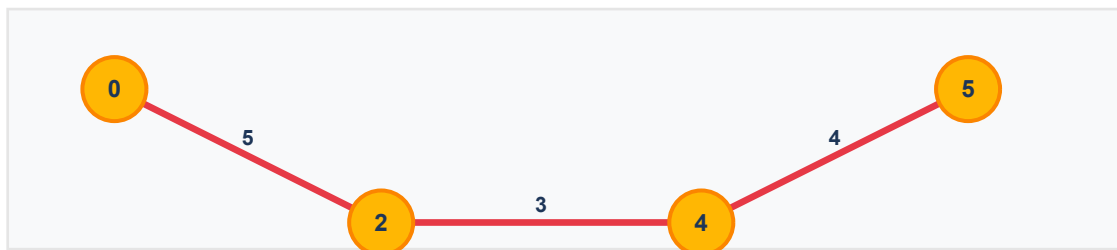
Đồ thị khảo sát có trọng số mẫu (6 đỉnh):



Bảng giải bằng tay từng bước giải thuật Dijkstra (bắt đầu từ đỉnh 0):

Bước	Đỉnh đã duyệt	1	2	3	4	5
1	0	4, 0	5, 0	inf, -	inf, -	inf, -
2	0, 1	-	5, 0	6, 1	11, 1	inf, -
3	0, 1, 2	-	-	6, 1	8, 2 (min(11, 5+3))	inf, -
4	0, 1, 2, 3	-	-	-	8, 2	14, 3 (6+8)
5	0, 1, 2, 3, 4	-	-	-	-	12, 4 (min(14, 8+4))
6	0, 1, 2, 3, 4, 5	4, 0	5, 0	6, 1	8, 2	12, 4

Sơ đồ đường đi ngắn nhất tối ưu tìm được:



Lộ trình chi tiết: 0 -> 2 -> 4 -> 5 với tổng chi phí ngắn nhất $d = 12$.

5. Thuật toán tìm Cây khung nhỏ nhất MST (Kruskal & Prim)

Khái niệm & Định nghĩa toán học: Cho đồ thị vô hướng liên thông có trọng số $G = (V, E, w)$. Cây khung (Spanning Tree) của G là một đồ thị con liên thông, chứa tất cả các đỉnh của G và không chứa chu trình. Cây khung nhỏ nhất (Minimum Spanning Tree - MST) là cây khung có tổng trọng số các cạnh của nó là nhỏ nhất. Một đồ thị có $|V|$ đỉnh thì bất kỳ cây khung nào cũng luôn có đúng $|V| - 1$ cạnh.

Để tìm MST, ta khảo sát hai giải thuật kinh điển dùng kỹ thuật tham lam (Greedy):

- Giải thuật Kruskal: Tiếp cận dựa trên việc chọn cạnh. Thuật toán sắp xếp toàn bộ cạnh của đồ thị theo trọng số tăng dần. Sau đó duyệt qua từng cạnh: nếu việc đưa cạnh này vào tập cạnh hiện tại không tạo ra chu trình, ta kết nạp nó vào MST. Quá trình dừng khi đã chọn đủ $|V| - 1$ cạnh.

Kỹ thuật kiểm tra chu trình: Dùng cấu trúc dữ liệu tập hợp rời rạc Disjoint Set Union (DSU) với các thao tác 'find' và 'union' có độ phức tạp gần như $O(1)$.

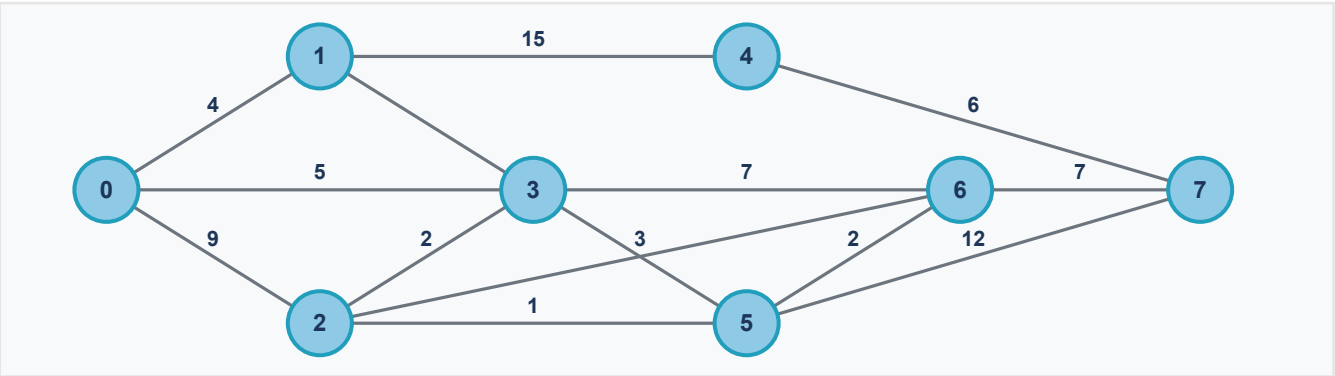
Độ phức tạp: $O(E \log E)$ thời gian do chi phí sắp xếp cạnh, $O(V)$ không gian để lưu trữ cây cha DSU.

- Giải thuật Prim: Tiếp cận dựa trên việc chọn đỉnh. Bắt đầu từ một đỉnh gốc bất kỳ. Thuật toán xây dựng cây khung bằng cách liên tục kết nạp cạnh có trọng số nhỏ nhất nối từ một đỉnh đã nằm trong cây (V_{MST}) tới một đỉnh chưa nằm trong cây ($V \setminus V_{MST}$). Quá trình lặp lại cho đến khi toàn bộ $|V|$ đỉnh được đưa vào cây.

Độ phức tạp: $O(V^2)$ thời gian khi dùng mảng thường hoặc $O(E \log V)$ khi dùng Hàng đợi ưu tiên. $O(V)$ không gian để lưu vết.

Nhận xét & So sánh thực tế: Kruskal chạy rất nhanh và trực quan trên đồ thị thưa (ít cạnh), đặc biệt thuận tiện khi danh sách cạnh đã được sắp xếp sẵn. Prim chạy hiệu quả hơn trên đồ thị dày (nhiều cạnh) khi dùng cấu trúc hàng đợi ưu tiên tối ưu.

Đồ thị khảo sát MST mẫu (8 đỉnh):



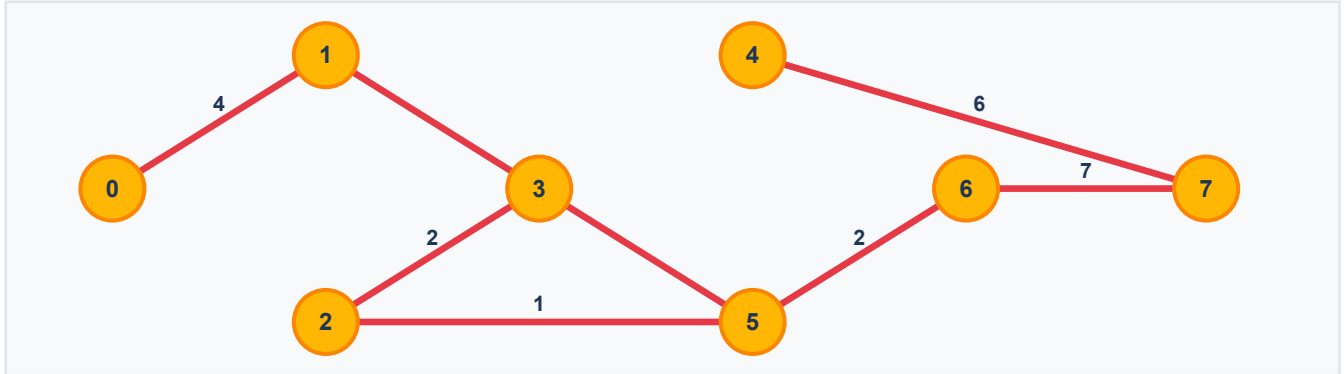
Thuật toán Kruskal (Giải bằng tay theo mẫu):

Khởi tạo ban đầu: MST = rỗng ($\emptyset$); $d(MST) = 0$; số đỉnh $n = 8$, số cạnh tối đa cần tìm là $n - 1 = 7$.

STT	Sắp xếp cạnh	Cây khung cực tiểu (MST)	Chọn / Loại	d (tổng trọng số)
1	(2,5) : 1	(2,5)	Chọn	1
2	(1,5) : 2	(2,5); (1,5)	Chọn	3
3	(2,3) : 2	(2,5); (1,5); (2,3)	Chọn	5
4	(5,6) : 2	(2,5); (1,5); (2,3); (5,6)	Chọn	7
5	(2,6) : 3	-	Loại (Tạo chu trình 2-5-6-2)	-
6	(0,1) : 4	(2,5); (1,5); (2,3); (5,6); (0,1)	Chọn	11
7	(0,3) : 5	-	Loại (Tạo chu trình 0-1-5-2-3-0)	-
8	(4,7) : 6	(2,5); (1,5); (2,3); (5,6); (0,1); (4,7)	Chọn	17
9	(3,6) : 7	-	Loại (Tạo chu trình 3-2-6-3)	-

10	(6,7) : 7	(2,5); (1,5); (2,3); (5,6); (0,1); (4,7); (6,7)	Chọn	24
11	(0,2) : 9	Dừng vòng lặp (break) vì đã đủ $ MST = n - 1 = 7$ cạnh	-	-
12	(5,7) : 12	-	-	-
13	(1,4) : 15	-	-	-

Sơ đồ Cây khung cực tiểu MST tìm được:



Tổng trọng số cây khung cực tiểu thu được: $d = 24$.

Thuật toán Prim (Dựa trên tập đỉnh & lân cận):

Khác với Kruskal sắp xếp cạnh tự do, Prim loang rộng cây khung từ một đỉnh gốc bắt đầu. Lần lượt kết nạp đỉnh ngoài có khoảng cách nhỏ nhất đến tập đỉnh hiện tại của cây khung (khởi đầu từ đỉnh 0):

Bư ớc	Đỉnh trong cây (V_{MST})	Cạnh ứng viên nối ra ngoài cây	Cạnh chọn	Trọng số
1	{0}	0-1 (4), 0-2 (9), 0-3 (5)	0 - 1	4
2	{0, 1}	0-2 (9), 0-3 (5), 1-4 (15), 1-5 (2)	1 - 5	2
3	{0, 1, 5}	0-2 (9), 0-3 (5), 5-2 (1), 5-6 (2), 5-7 (12)	5 - 2	1
4	{0, 1, 2, 5}	0-3 (5), 2-3 (2), 5-6 (2), 2-6 (3), 5-7 (12)	2 - 3	2
5	{0, 1, 2, 3, 5}	5-6 (2), 2-6 (3), 3-6 (7), 5-7 (12)	5 - 6	2
6	{0, 1, 2, 3, 5, 6}	6-7 (7), 5-7 (12)	6 - 7	7
7	{0..7 - trừ 4}	7-4 (6), 1-4 (15)	7 - 4	6
KQ	Tất cả 8 đỉnh đã được đưa vào cây.	-	Đầy đủ cây khung	Tổng trọng số = 24 (khớp Kruskal)

6. Thuật toán luồng cực đại (Max Flow)

Khái niệm & Định nghĩa toán học: Cho mạng luồng (Flow Network) là một đồ thị có hướng $G = (V, E)$ trong đó mỗi cạnh (u, v) có dung lượng (Capacity) $c(u, v) \geq 0$. Mạng có một đỉnh nguồn (Source) s là nơi phát luồng, và một đỉnh đích (Sink) t là nơi tiếp nhận luồng. Luồng f trên mạng G là một hàm gán cho mỗi cạnh một giá trị số thực thỏa mãn hai điều kiện:

- Ràng buộc dung lượng: $0 \leq f(u, v) \leq c(u, v)$ với mọi cạnh.
- Bảo toàn luồng: Tổng luồng đi vào một đỉnh bằng tổng luồng đi ra khỏi đỉnh đó (với mọi đỉnh khác s và t).

Định lý Luồng cực đại - Lát cắt cực tiểu (Max-Flow Min-Cut Theorem): Giá trị của luồng cực đại đi từ s tới t luôn bằng với dung lượng nhỏ nhất của một lát cắt $(s, t\text{-cut})$ phân hoạch tập đỉnh thành hai phần chứa s và t .

Nguyên lý hoạt động thuật toán Ford-Fulkerson & Edmonds-Karp:

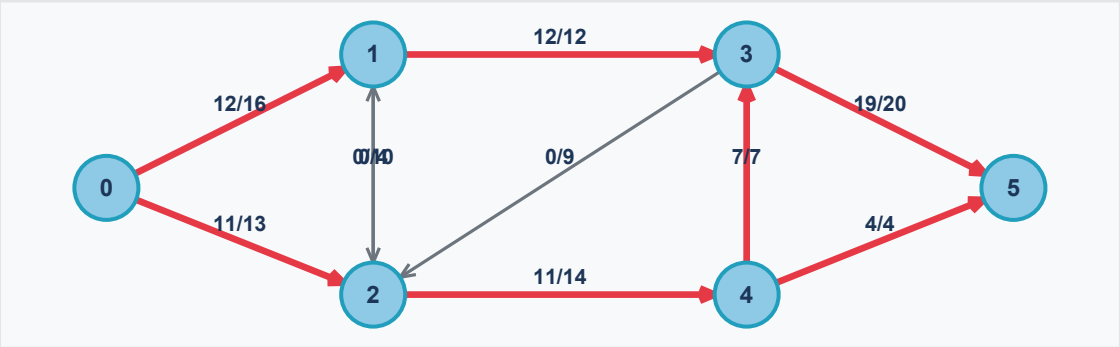
Thuật toán hoạt động trên Đồ thị dư (Residual Graph) G_f . Mỗi cạnh có hướng (u, v) trên G_f biểu thị khả năng gửi thêm luồng theo chiều xuôi $c_f(u, v) = c(u, v) - f(u, v)$, và khả năng gửi trả lại luồng theo chiều ngược $c_f(v, u) = f(u, v)$. Các bước thực hiện:

- Sử dụng giải thuật duyệt đồ thị để tìm một đường đi đơn từ s tới t trên đồ thị dư G_f có dung lượng dư lớn hơn 0 (Đường tăng luồng - Augmenting Path). Giải thuật Edmonds-Karp sử dụng cụ thể giải thuật BFS để tìm đường đi ngắn nhất (ít cạnh nhất), giúp giới hạn số lần lặp tối đa $O(VE)$.
- Tìm bottleneck (df) là dung lượng dư nhỏ nhất của các cạnh trên đường đi đó.
- Tăng luồng thực tế dọc theo đường tăng luồng thêm df (tăng luồng cạnh xuôi, giảm luồng cạnh ngược trên đồ thị dư). Lặp lại cho đến khi không còn đường đi từ s đến t .

Độ phức tạp thuật toán: $O(V E^2)$ thời gian đối với giải thuật Edmonds-Karp, độc lập với giá trị luồng tối đa, khắc phục nhược điểm lặp vô hạn của Ford-Fulkerson trên số thực.

Minh họa đồ thị luồng tối ưu (Edmonds-Karp):

Nhãn trên cạnh biểu thị tỷ lệ luồng/dung lượng thực tế (flow/capacity). Đỉnh 0 là nguồn (Source), Đỉnh 5 là đích (Sink). Cạnh tô màu đỏ biểu thị các cạnh có luồng đi qua.



Lần lặp	Đường tăng luồng tìm thấy (BFS)	Dung lượng dư nhỏ nhất (Bottleneck)	Tổng luồng tăng lũy tiến
1	0 -> 1 -> 3 -> 5	$\min(16, 12, 20) = 12$	12
2	0 -> 2 -> 4 -> 5	$\min(13, 14, 4) = 4$	$12 + 4 = 16$
3	0 -> 2 -> 4 -> 3 -> 5 (Duyệt qua cạnh dư 4-3 còn 7, 3-5 còn 8)	$\min(9, 10, 7, 8) = 7$	$16 + 7 = 23$
4	Không còn đường tăng luồng từ 0 đến 5 trên mạng dư. Kết thúc.	-	Luồng cực đại = 23

7. Đường đi và chu trình Euler: Fleury vs Hierholzer

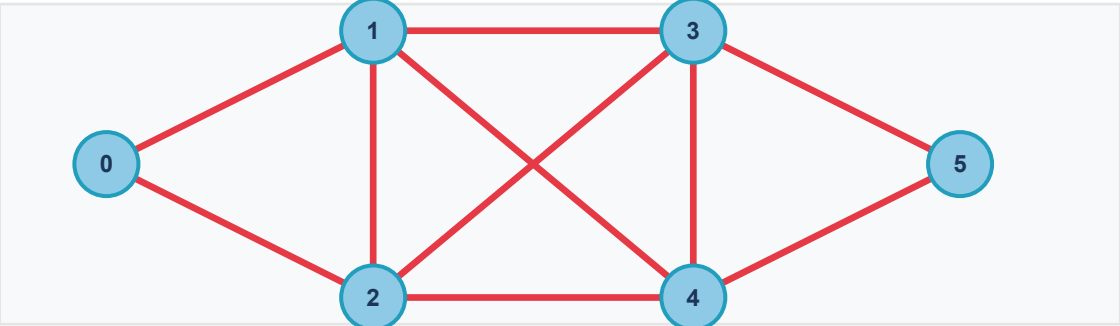
Khái niệm toán học & Điều kiện tồn tại:

- Đường đi Euler (Eulerian Path): Đường đi đi qua mỗi cạnh của đồ thị đúng một lần.
- Chu trình Euler (Eulerian Circuit): Đường đi Euler xuất phát và kết thúc tại cùng một đỉnh.
- Định lý Euler vô hướng: Đồ thị vô hướng liên thông G có chu trình Euler khi và chỉ khi tất cả các đỉnh của đồ thị đều có bậc chẵn. Đồ thị có đường đi Euler khi và chỉ khi có đúng 0 hoặc 2 đỉnh bậc lẻ.

Để tìm chu trình Euler, ta có hai giải thuật điển hình với phương thức tiếp cận và độ phức tạp khác nhau:

- Giải thuật Fleury:** Thực hiện đi tham lam. Xuất phát tại một đỉnh, ở mỗi bước ta chọn một cạnh kề chưa đi qua để đi tiếp theo quy tắc: Tránh đi qua cạnh cầu (Bridge) của đồ thị còn lại trừ khi không còn cạnh nào khác để chọn (cạnh cầu là cạnh mà nếu xóa đi sẽ làm tăng số lượng thành phần liên thông của đồ thị).
Độ phức tạp: $O(E^2)$ thời gian do mỗi lần chọn cạnh phải chạy thuật toán BFS/DFS để xác định xem cạnh đó có là cầu hay không.
- Giải thuật Hierholzer:** Hoạt động dựa trên việc lồng ghép các chu trình con đơn giản. Bắt đầu từ một đỉnh, đi tự do theo các cạnh chưa đi cho đến khi quay về đỉnh gốc tạo thành một chu trình đơn, xóa các cạnh này và đẩy lộ trình vào một Ngăn xếp (Stack). Do tính chất bậc chẵn, nếu đồ thị còn cạnh chưa đi, ta tìm một đỉnh kề trên chu trình đã có còn cạnh chưa xét, tiếp tục đi một chu trình mới rồi ghép trực tiếp vào chu trình ban đầu bằng cơ chế pop Stack.
Độ phức tạp: $O(E)$ thời gian và không gian cực kỳ tối ưu vì mỗi cạnh chỉ bị duyệt và xóa đúng một lần duy nhất.

Minh họa đồ thị có chu trình Euler (Độ chẵn của bậc):



Đặc điểm	Giải thuật Fleury (7.4)	Giải thuật Hierholzer (7.5)
Nguyên lý chính	Đi tham lam, tránh đi qua cạnh 'cầu' (bridge) của phần đồ thị còn lại trừ khi không có lựa chọn khác.	Tìm các chu trình con đơn giản kề nhau và lồng chúng lại với nhau thông qua cơ chế Stack.
Kiểm tra cầu	Có. Đòi hỏi chạy DFS ở mỗi bước để đếm số thành phần liên thông khi thử xóa cạnh.	Không. Chỉ cần quản lý danh sách cạnh kề và xóa cạnh trực tiếp.
Độ phức tạp	$O(E^2)$	$O(E)$ - Rất nhanh và tối ưu
Kết quả lộ trình mẫu	0 -> 1 -> 2 -> 3 -> 1 -> 4 -> 3 -> 5 -> 4 -> 2 -> 0	0 -> 1 -> 2 -> 3 -> 1 -> 4 -> 3 -> 5 -> 4 -> 2 -> 0

Bảng chạy bằng tay từng bước giải thuật Tìm chu trình Euler:

TH1: Chạy vết giải thuật Hierholzer (sử dụng Ngăn xếp Stack):

Bư ớc	Đỉnh U	Cạnh đi tiếp khả dụng	Hành động	Stack (Ngăn xếp)	Chu trình Euler (Circuit)
0	0	Tất cả	Khởi tạo Stack	[0]	[]
1	0	(0,1), (0,2)	Đi 1, xóa cạnh (0,1)	[0, 1]	[]
2	1	(1,2), (1,3), (1,4)	Đi 2, xóa cạnh (1,2)	[0, 1, 2]	[]
3	2	(2,0), (2,3), (2,4)	Đi 0, xóa cạnh (2,0)	[0, 1, 2, 0]	[]
4	0	Không còn	Pop 0 đưa vào Circuit	[0, 1, 2]	[0]
5	2	(2,3), (2,4)	Đi 3, xóa cạnh (2,3)	[0, 1, 2, 3]	[0]
6	3	(3,1), (3,4), (3,5)	Đi 1, xóa cạnh (3,1)	[0, 1, 2, 3, 1]	[0]
7	1	(1,4)	Đi 4, xóa cạnh (1,4)	[0, 1, 2, 3, 1, 4]	[0]
8	4	(4,3), (4,5)	Đi 3, xóa cạnh (4,3)	[0, 1, 2, 3, 1, 4, 3]	[0]
9	3	(3,5)	Đi 5, xóa cạnh (3,5)	[0, 1, 2, 3, 1, 4, 3, 5]	[0]
10	5	(5,4)	Đi 4, xóa cạnh (5,4)	[0, 1, 2, 3, 1, 4, 3, 5, 4]	[0]
11	4	(4,2)	Đi 2, xóa cạnh (4,2)	[0, 1, 2, 3, 1, 4, 3, 5, 4, 2]	[0]
12	2	Không còn	Pop 2 đưa vào Circuit	[0, 1, 2, 3, 1, 4, 3, 5, 4]	[0, 2]
13-2 1	-	-	Pop dần toàn bộ Stack còn lại	[]	[0, 2, 4, 5, 3, 4, 1, 3, 2, 1, 0]

TH2: Chạy vết giải thuật Fleury (Tránh chọn cạnh cầu - bridge):

Bư ớc	Đỉnh U	Các cạnh đi tiếp khả dụng	Trạng thái Cầu (Bridge) của các cạnh kề	Cạnh chọn	Lộ trình Euler lũy tiến
1	0	(0,1), (0,2)	Không có cạnh cầu	Chọn (0,1), xóa	0 -> 1
2	1	(1,2), (1,3), (1,4)	Không có cạnh cầu	Chọn (1,2), xóa	0 -> 1 -> 2
3	2	(2,0), (2,3), (2,4)	Không có cạnh cầu	Chọn (2,0), xóa	0 -> 1 -> 2 -> 0
4	0	(0,2)	Chỉ còn 1 cạnh (bắt buộc chọn)	Chọn (0,2), xóa	0 -> 1 -> 2 -> 0 -> 2
5	2	(2,3), (2,4)	Không có cạnh cầu	Chọn (2,4), xóa	... -> 2 -> 4
6	4	(4,1), (4,3), (4,5)	Không có cạnh cầu	Chọn (4,5), xóa	... -> 4 -> 5
7	5	(5,3)	Chỉ còn 1 cạnh (bắt buộc chọn)	Chọn (5,3), xóa	... -> 5 -> 3
8	3	(3,1), (3,4), (3,2)	Không có cạnh cầu	Chọn (3,4), xóa	... -> 3 -> 4
9	4	(4,1)	Chỉ còn 1 cạnh	Chọn (4,1), xóa	... -> 4 -> 1
10	1	(1,3)	Chỉ còn 1 cạnh	Chọn (1,3), xóa	... -> 1 -> 3
11	3	(3,2)	Chỉ còn 1 cạnh	Chọn (3,2), xóa	... -> 3 -> 2
12	2	Không còn	Hoàn thành chu trình	-	0->1->2->0->2->4->5->3->4->1->3->2 (khớp Euler)